

Sri Lanka Institute of Information Technology



Lab Submission

<Lab Sheet No-3>

Artificial Intelligence and Machine Learning - IT2011

2025.08.19

Year 02 Semester 1

IT24103015- Jayawardhane T D

Task 01

```
2s  import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set(color_codes=True)
```

```
36s  from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Figure 1: connecting with Drive

The screenshot shows the Google Colab interface. On the left is a file explorer with a sidebar containing icons for files, folders, and search. The main area shows a file tree with folders named 'drive' and 'sample_data'. On the right, a code editor shows two code cells. The first cell, labeled '[4]', contains the code to import 'drive' from 'google.colab' and mount it at '/content/drive'. Below the code, it says 'Mounted at /content/drive'. The second cell, labeled '[11]', contains the code to read a CSV file from the drive: 'df= pd.read_csv('/content/drive/MyDrive/cars_data.csv')'.

Figure 2: uploading card_data.csv file

```
0s [12] display(df.head())
```

	Make	Model	Year	Engine Fuel Type	Engine HP	Engine Cylinders	Transmission Type	Driven_Wheels	Number of Doors	Market Category	Vehicle Size
0	BMW	Series M	2011	premium unleaded (required)	335.0	6.0	MANUAL	rear wheel drive	2.0	Factory Tuner,Luxury,High- Performance	Compact
1	BMW	Series	2011	premium unleaded (required)	300.0	6.0	MANUAL	rear wheel drive	2.0	Luxury,Performance	Compact
2	BMW	Series	2011	premium unleaded (required)	300.0	6.0	MANUAL	rear wheel drive	2.0	Luxury,High- Performance	Compact
3	BMW	Series	2011	premium unleaded (required)	230.0	6.0	MANUAL	rear wheel drive	2.0	Luxury,Performance	Compact
4	BMW	Series	2011	premium unleaded (required)	230.0	6.0	MANUAL	rear wheel drive	2.0	Luxury	Compact

Figure 3: Display the first 5 rows

0s  `display(df.tail())`



	Make	Model	Year	HP	Total Cylinders	Mechanism	Drive Mode	MPG-H	MPG-C	Price
11909	Acura	ZDX	2012	300.0	6.0	AUTOMATIC	all wheel drive	23	16	46120
11910	Acura	ZDX	2012	300.0	6.0	AUTOMATIC	all wheel drive	23	16	56670
11911	Acura	ZDX	2012	300.0	6.0	AUTOMATIC	all wheel drive	23	16	50620
11912	Acura	ZDX	2013	300.0	6.0	AUTOMATIC	all wheel drive	23	16	50920
11913	Lincoln	Zephyr	2006	221.0	6.0	AUTOMATIC	front wheel drive	26	17	28995

[+ Code](#) [+ Text](#)

Figure 4: Display last 5 rows

0s  `df.dtypes`



Make	object
Model	object
Year	int64
Engine Fuel Type	object
Engine HP	float64
Engine Cylinders	float64
Transmission Type	object
Driven_Wheels	object
Number of Doors	float64
Market Category	object
Vehicle Size	object
Vehicle Style	object
highway MPG	int64
city mpg	int64
Popularity	int64
MSRP	int64

Figure 5: Check for the Types of the Data

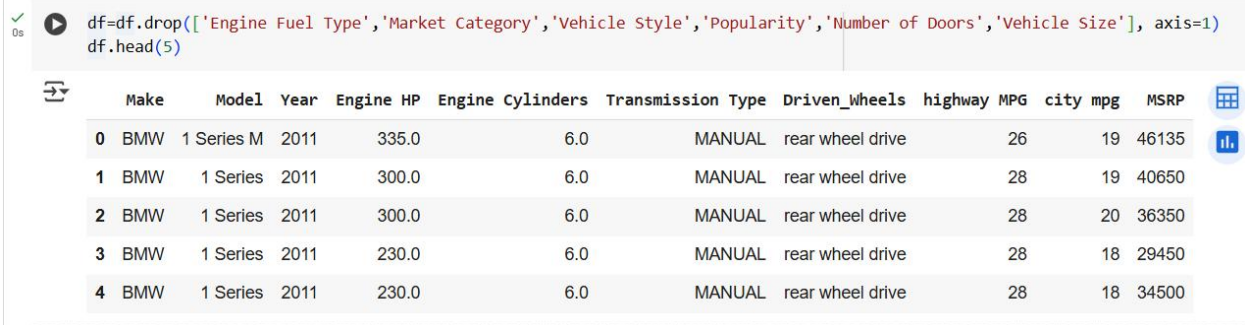


Figure 6: Remove the irrelevant columns

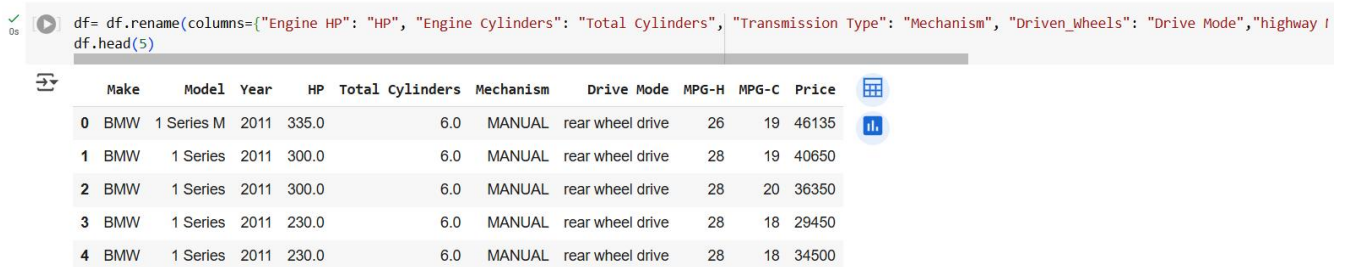


Figure 7: Renaming the columns

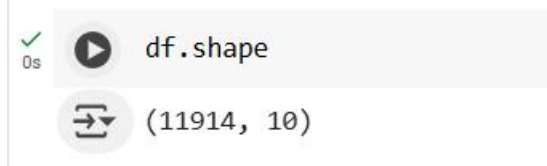


Figure 8: Check the dimension of the DataFrame

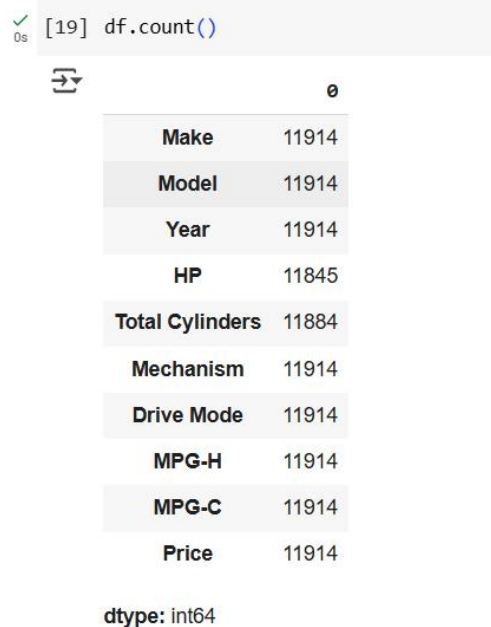


Figure 9: count no of rows

```
✓ 0s [1] duplicate_rows_df = df[df.duplicated()]
      print("number of duplicate rows: ", duplicate_rows_df.shape)

⇅ number of duplicate rows: (989, 10)
```

Figure 10: Find Duplicated rows

```
✓ 0s [21] df= df.drop_duplicates()
      df.head(5)
```

	Make	Model	Year	HP	Total Cylinders	Mechanism	Drive Mode	MPG-H	MPG-C	Price
0	BMW	1 Series M	2011	335.0	6.0	MANUAL	rear wheel drive	26	19	46135
1	BMW	1 Series	2011	300.0	6.0	MANUAL	rear wheel drive	28	19	40650
2	BMW	1 Series	2011	300.0	6.0	MANUAL	rear wheel drive	28	20	36350
3	BMW	1 Series	2011	230.0	6.0	MANUAL	rear wheel drive	28	18	29450
4	BMW	1 Series	2011	230.0	6.0	MANUAL	rear wheel drive	28	18	34500

Figure 11: Remove the duplicate rows

```

✓ 0s print(df.isnull().sum())

```

Make	0
Model	0
Year	0
HP	69
Total Cylinders	30
Mechanism	0
Drive Mode	0
MPG-H	0
MPG-C	0
Price	0

dtype: int64

```

✓ 0s [24] df= df.dropna()
      df.count()

```

	0
Make	10827
Model	10827
Year	10827
HP	10827
Total Cylinders	10827
Mechanism	10827
Drive Mode	10827
MPG-H	10827
MPG-C	10827
Price	10827

dtype: int64

Figure 12: Drop the Missing or Null Values

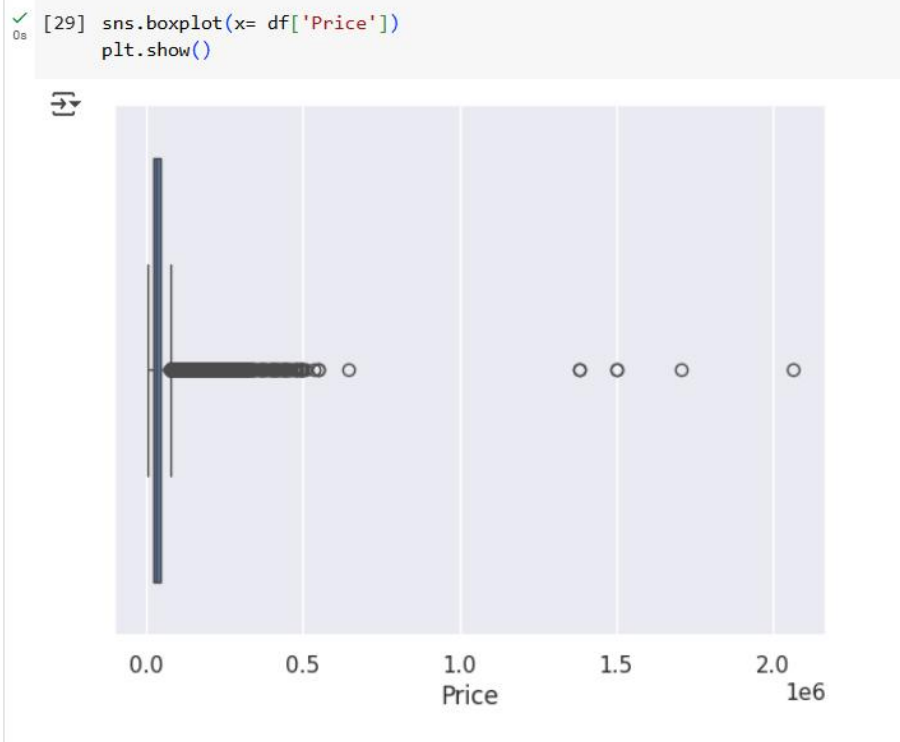


Figure 13: Outlier detection for price

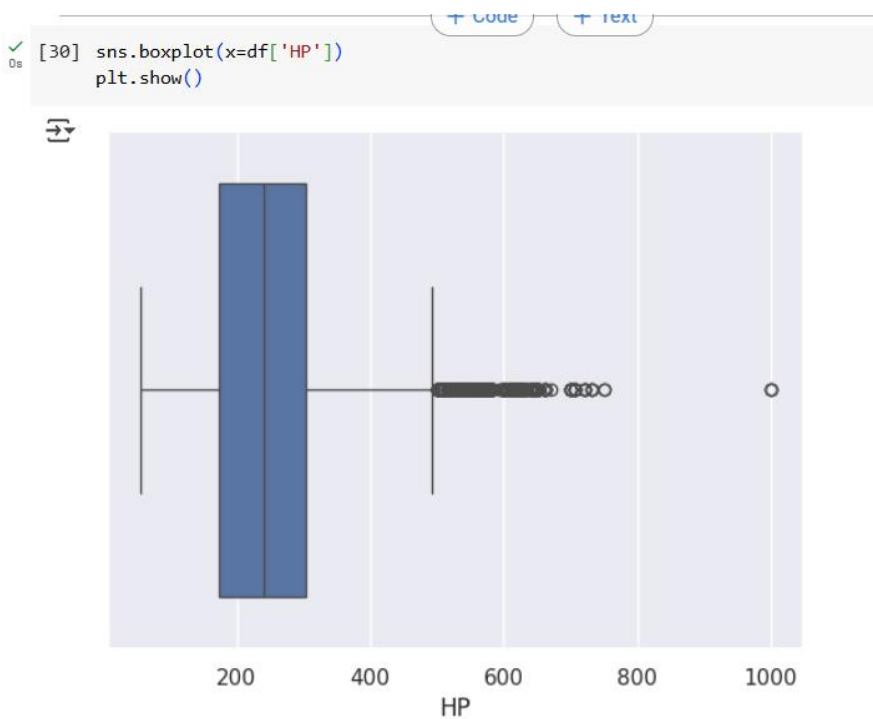


Figure 14: Outlier detection for HP

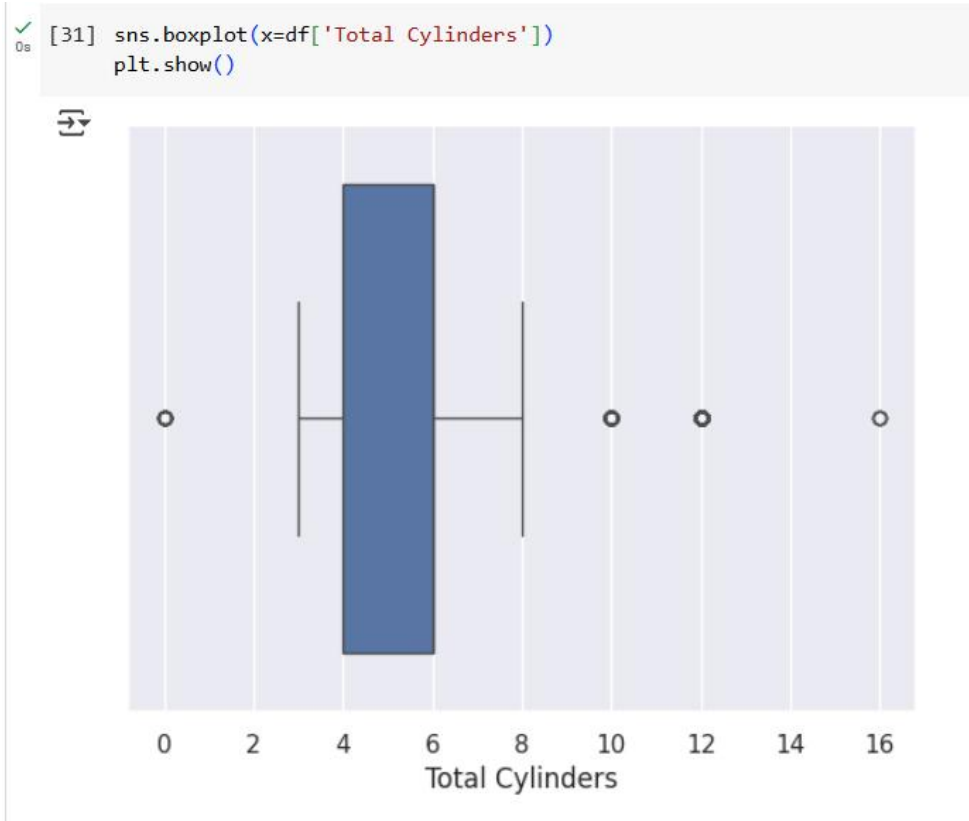


Figure 15: Outlier detection for Total Cylinders

```
numeric_cols = df.select_dtypes(include='number').columns

Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

condition = ~((df[numeric_cols] < (Q1 - 1.5 * IQR)) | (df[numeric_cols] > (Q3 + 1.5 * IQR))).any(axis=1)

df_cleaned = df[condition]
print(df_cleaned.shape)
```

(9191, 10)

Figure 16: Quartiles

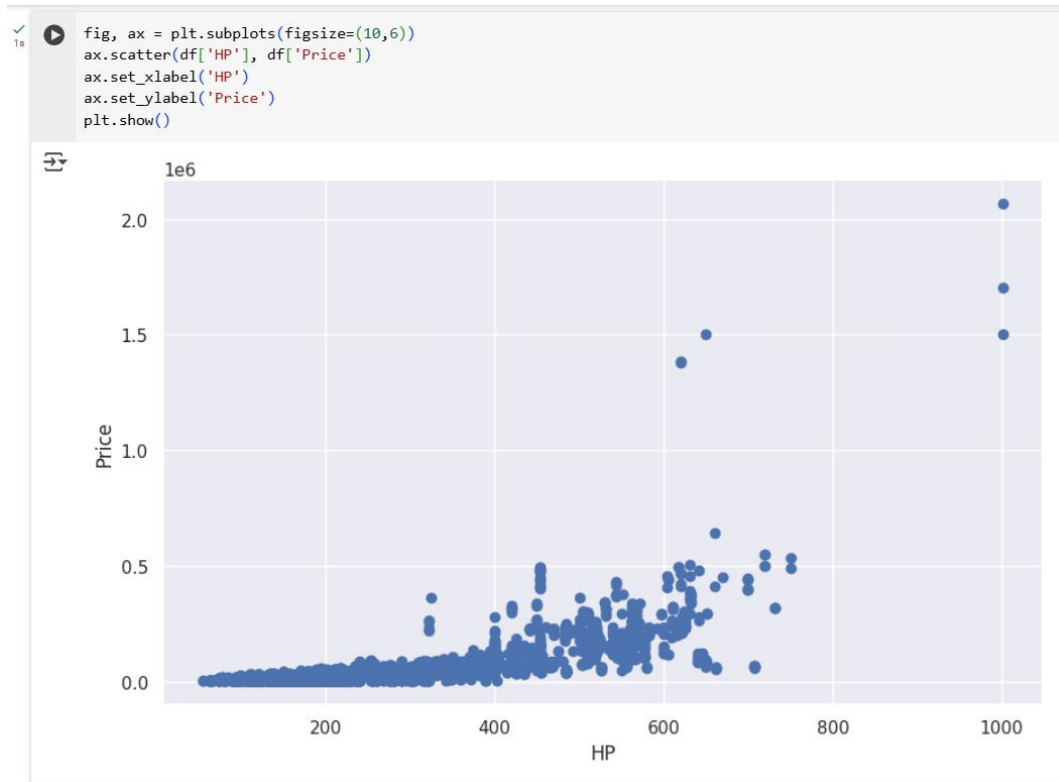


Figure 17:Scatter Plot

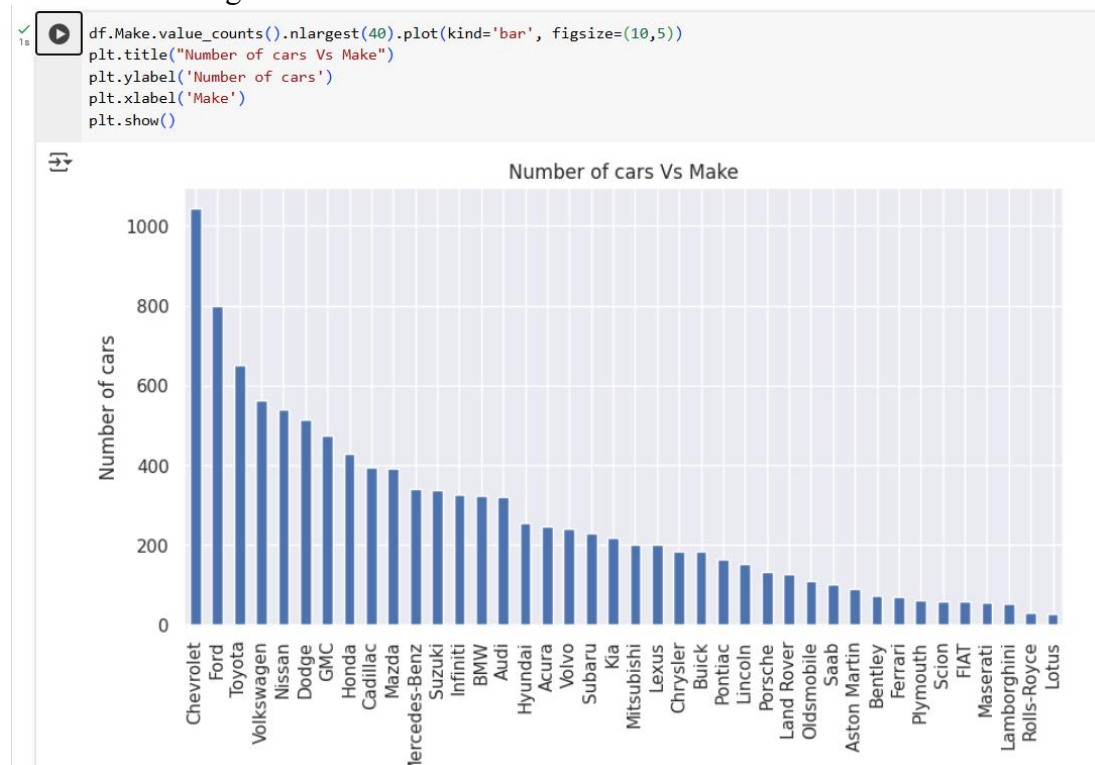


Figure 18:Histogram

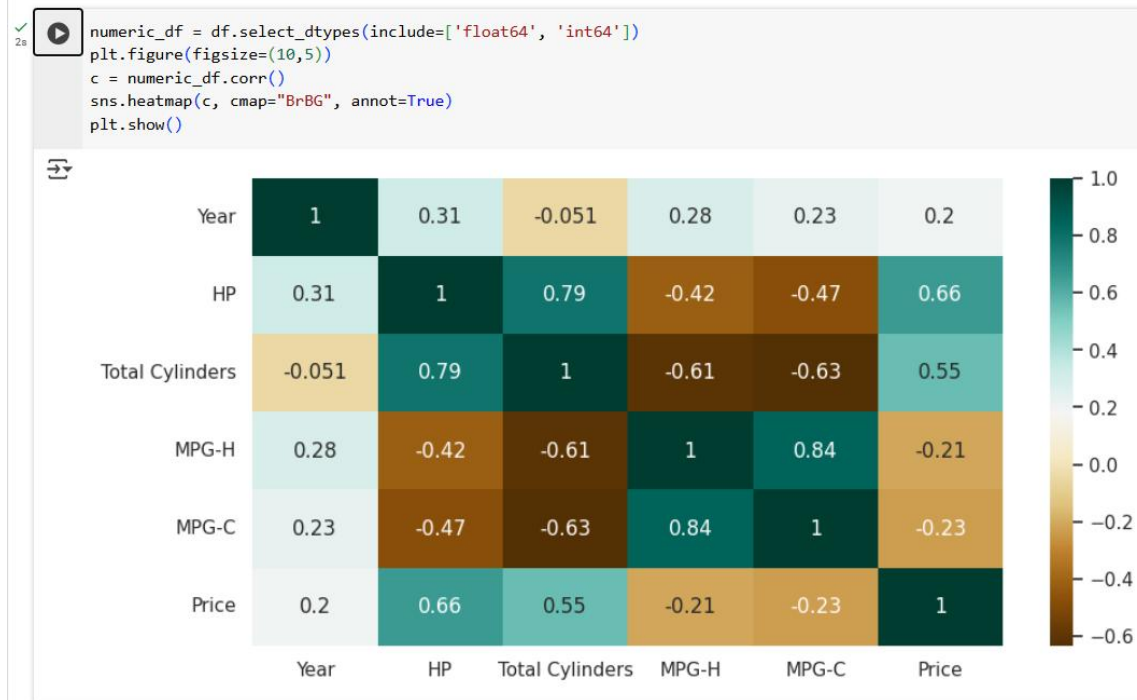


Figure 19: Heat Map

Task 2

```

import pandas as pd

[2] df = pd.read_csv("student_data.csv")

[3] print("First 5 rows:")
    print(df.head()) # shows first 5 rows

```

Figure 20- task2.1



Figure 21: first 5 rows



Figure 22: last 5 rows

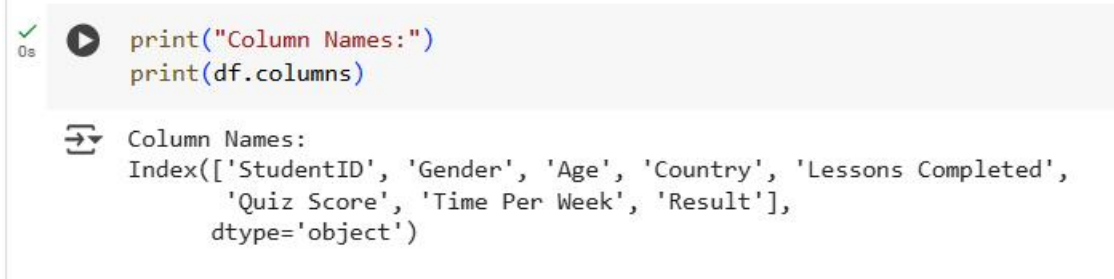


Figure 23: column names

```

✓ 0s [6] df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   StudentID             20 non-null    object  
1   Gender                 20 non-null    object  
2   Age                   18 non-null    float64  
3   Country                20 non-null    object  
4   Lessons Completed     20 non-null    int64  
5   Quiz Score             18 non-null    float64  
6   Time Per Week          20 non-null    int64  
7   Result                 20 non-null    object  
dtypes: float64(2), int64(2), object(4)
memory usage: 1.4+ KB

```

Figure 24:summary of the DataFrame

```

✓ 0s [7] df.describe()

```

	Age	Lessons Completed	Quiz Score	Time Per Week
count	18.000000	20.000000	18.000000	20.000000
mean	21.722222	22.350000	62.194444	8.550000
std	2.136670	7.596918	25.133256	3.316228
min	18.000000	10.000000	20.000000	3.000000
25%	20.000000	17.250000	41.250000	6.000000
50%	21.500000	23.500000	71.000000	8.500000
75%	23.000000	28.250000	83.500000	10.250000
max	26.000000	35.000000	95.000000	15.000000

Figure 25:Descriptive Statistical Features of the DataFrame

```

✓ 0s [11] df.fillna(df['Age'].mean(), inplace=True)
        df.fillna(df['Quiz Score'].mean(), inplace=True)

```

Figure 26;Handling Missing Data

```

✓ [12] 0s
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})

df["Result"] = df["Result"].map({"Pass": 1, "Fail": 0})

✓ [13] 0s
df["Engagement Level"] = df["Lessons Completed"] / (df["Time Per Week"] + 0.0001)

```

Figure 27: Encode Categorical Data, Add a New Feature “EngagementLevel”

```

✓ [14] 1s
# import MinMaxScaler Library to complete this task
from sklearn.preprocessing import MinMaxScaler

## Normalization
scaler = MinMaxScaler()

scale_cols = ["Age", "Lessons Completed", "Quiz Score", "Time Per Week", "Engagement Level"]

df[scale_cols] = scaler.fit_transform(df[scale_cols])

df.head()

```

	StudentID	Gender	Age	Country	Lessons Completed	Quiz Score	Time Per Week	Result	Engagement Level
0	S001	1	0.250	India	0.60	0.780000	0.583333	1	0.388892
1	S002	0	0.500	USA	0.80	0.826667	0.750000	1	0.388894
2	S003	1	0.125	Sri Lanka	0.32	0.333333	0.416667	0	0.246295
3	S004	0	0.375	India	0.72	0.906667	0.500000	1	0.737453
4	S005	0	0.000	USA	0.20	0.200000	0.166667	0	0.674063

Figure 28: Normalize numerical features using MinMaxScaler